ReMix: reparameterization for mixtures

Proof of Concept

Команда проекта: Александр Уденеев, Алексей Пономарев,
Лина Горбунова, Максим Иванов

ИАД, МФТИ

Этап прототипа

Цель этапа

Реализовать один простой оценщик в интерфейсе будущей библиотеки и подготовить небольшой воспроизводимый эксперимент.

1) Представить смесь независимо от семейства компонент.
2) Получить скалярную функцию потерь для `backward()`.
3) Сохранить общий интерфейс для следующих оценщиков.

Реализован ScoreFunction как простой пример оценщика

Повторение про область задачи

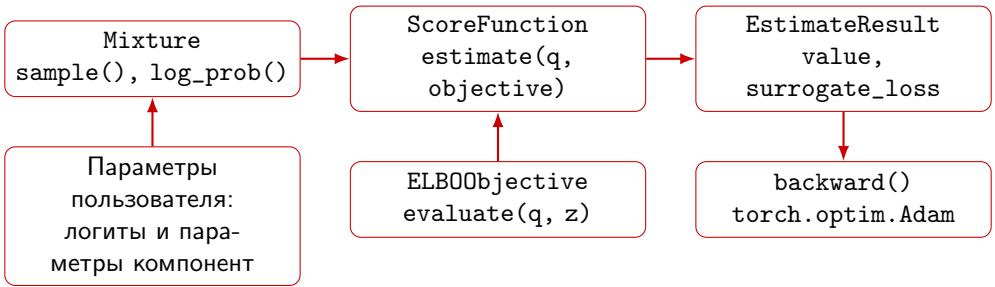
$$q_\theta(z) = \sum_{k=1}^{K} \pi_k q_{k,\phi_k}(z), \qquad \pi = \mathrm{softmax}(a), \quad \theta = (a, \phi_1, \ldots, \phi_K).$$

Условия совместимости

1) Одинаковый shape события.
2) Общая базовая мера.
3) Согласованные устройства и типы.
4) Компоненты без batch-измерений.

Оценщик принимает `Mixture` — отдельное распределение отклоняется.

Архитектура текущей реализации



После шага оптимизатора заново создаётся смесь с текущими параметрами.

`src/remix/`

Как представлена смесь

`Mixture(logits, components)` хранит список распределений PyTorch.

$$\ell_k(z) = \log \pi_k + \log q_k(z), \qquad \log q(z) = \operatorname{logsumexp}_k \ell_k(z).$$

$$r_k(z) = \operatorname{softmax}_k \ell(z) = \frac{\pi_k q_k(z)}{q(z)}.$$

- `log_prob`: устойчивая агрегация плотностей компонент.
- `responsibilities`: вероятности принадлежности компонентам.
- `sample`: выбор индекса из Categorical и значения компоненты.

Генерируются кандидаты от всех компонент, затем выбирается один.

`src/remix/distributions.py`

Целевая функция

$$\mathcal{L}(\theta) = \mathbb{E}_{z \sim q_\theta}[f_\theta(z)], \qquad f_\theta(z) = \log p(x, z) - \log q_\theta(z).$$

`ELBOObjective(log_joint)`

Принимает пользовательскую функцию $\log p(x, z)$. Метод `evaluate(q, z)` возвращает значения $f_\theta(z)$.

Поле результата	Назначение
`value`	Среднее $f_\theta(z)$ без градиента
`surrogate_loss`	Скаляр для минимизации через `backward()`

В бенчмарке p — нормированное целевое распределение: $\mathcal{L}(\theta) = -\,\mathrm{KL}(q_\theta \| p)$.

`src/remix/objectives.py`

Score-function

При допустимости дифференцирования под интегралом:

$$\nabla_\theta \mathcal{L} = \mathbb{E}_{q_\theta}[f_\theta(z)\nabla_\theta \log q_\theta(z) + \partial_\theta f_\theta(z)]\,.$$

$$\widehat{g} = \frac{1}{N}\sum_{i=1}^{N}[(f_\theta(z_i) - b)\nabla_\theta \log q_\theta(z_i) + \partial_\theta f_\theta(z_i)]\,, \quad z_i \sim q_\theta.$$

- Производные вычисляются при фиксированных z_i.
- b — фиксированный скаляр, независимый от текущих сэмплов.
- $\mathbb{E}_q[\nabla \log q] = 0$: такой baseline не меняет ожидание оценки.

Почему это удобная базовая реализация

Нужен дифференцируемый `log_prob` — производная через выбор компоненты и `rsample` не требуются.

`src/remix/estimators.py`

Как нужный градиент получает autograd

```
z = q.sample(torch.Size([self.n_samples])).detach()
log_q = q.log_prob(z)
values = objective.evaluate(q, z)

surrogate = (values.detach() - self.baseline) * log_q + values
return EstimateResult(values.detach().mean(), -surrogate.mean())
```

$$S(\theta) = \frac{1}{N} \sum_i \left[(\mathrm{stopgrad}(f_\theta(z_i)) - b) \log q_\theta(z_i) + f_\theta(z_i) \right], \qquad \nabla_\theta S = \widehat{g}.$$

- `detach()` у веса исключает лишнее слагаемое производной.
- Минус перед средним переводит максимизацию в минимизацию.

`src/remix/estimators.py`

Пользовательский цикл обучения

```
components = [Normal(-2.0, 1.0), Laplace(2.0, 1.0)]
logits = torch.nn.Parameter(torch.zeros(2))
target = Mixture(torch.tensor([0.2, 0.8]).log(), components)
objective = ELBOObjective(target.log_prob)
estimator = ScoreFunction(n_samples=512)
optimizer = torch.optim.Adam([logits], lr=0.05)

for step in range(300):
    optimizer.zero_grad()
    q = Mixture(logits, components)
    result = estimator.estimate(q, objective)
    result.surrogate_loss.backward()
    optimizer.step()
```

Здесь обучаются только веса — параметры компонент фиксированы.

`examples/benchmark.py`

Бенчмарк

Сценарий	Компонента 1	Компонента 2
Гауссовская смесь	$\mathcal{N}(-2, 1)$	$\mathcal{N}(2, 1)$
Смесь Лапласа	$\mathrm{Laplace}(-2, 1)$	$\mathrm{Laplace}(2, 1)$
Разные семейства	$\mathcal{N}(-2, 1)$	$\mathrm{Laplace}(2, 1)$

- Начальные веса $(0.5, 0.5)$, целевые веса $(0.2, 0.8)$.
- Adam: шаг 0.05, 300 итераций, 512 сэмплов на итерацию.
- Seed: 42; оценка KL по 10 000 сэмплам до и после обучения.

$$\widehat{\mathrm{KL}}(q\|p) = \frac{1}{M}\sum_{j=1}^{M}[\log q(z_j) - \log p(z_j)], \quad z_j \sim q.$$

Результат бенчмарка

```
Normal mixture: estimated KL 0.2001 -> 0.0000
Learned weights: [0.19305355846881866, 0.8069464564323425]

Laplace mixture: estimated KL 0.1624 -> 0.0003
Learned weights: [0.1880115270614624, 0.8119884729385376]

Normal + Laplace mixture: estimated KL 0.1755 -> 0.0001
Learned weights: [0.191883847117424, 0.8081161975860596]
```

Вывод

Результат текущего этапа

Написан минимальный прототип: смесь $\rightarrow$ ELBO $\rightarrow$ score-function $\rightarrow$ стандартный оптимизатор PyTorch.

Дальнейшие планы

Далее будем добавлять следующие оценщики через существующий интерфейс.